Troubleshoot Your Code

Project Report: Roast Toast Coast Café Website
IBM SkillsBuild Lab: Troubleshoot Your Code Using IBM Bob
Prepared by: Prateeksha Udayan | October 2026

1. Objective

The goal of this lab is to find and correct errors in a small website, and to understand how each code change affects the way the page works. The lab covers analyzing source code for syntax and functional errors, writing structured prompts for troubleshooting, reviewing recommended fixes, and comparing the original and corrected code.

2. Scenario

The website belongs to a fictional business, Roast Toast Coast Café. It has a welcome message, a short tagline, a background image, and a button labeled **Our Menu** that reveals the menu items when clicked. The page is a single file (`Menu.html`) written in HTML, CSS and JavaScript.

3. Problem Observed

When the original `Menu.html` was opened in Microsoft Edge, the page showed only a blank black screen. The welcome box, the tagline, the **Our Menu** button and the image were not visible.

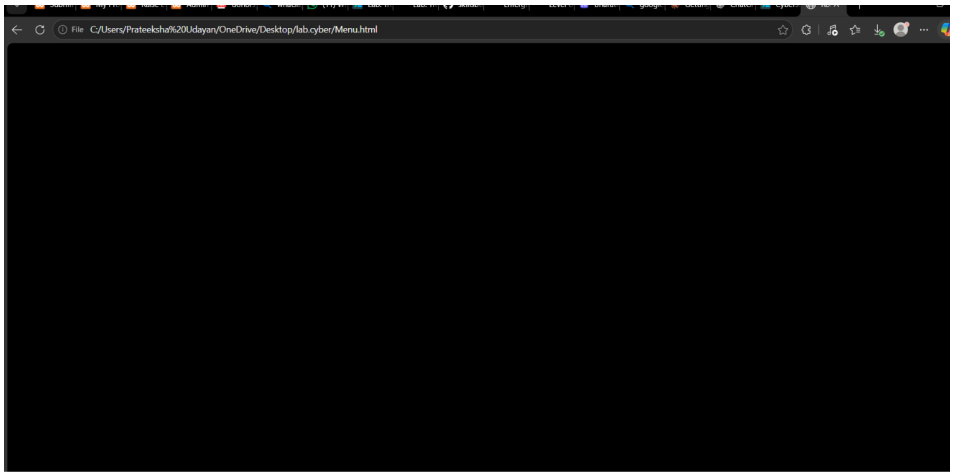


Figure 1: Original page, blank black screen.

4. Error Analysis

Two errors were found in the original code.

#	Location	Type	Cause and effect	Fix
1	CSS, `.image-container`, `background-image` line	Syntax error	The `url('...')` was missing its closing parenthesis. The browser treated the rest of the stylesheet as part of the unfinished value and ignored it. All later styling was lost, so the text became black on a black background and the image did not load.	Add the missing `)` after the image URL.
2	JavaScript, `toggleMenuItems()`	Functional error	The function used `querySelector('.menu-item')`, but the list's class is `menu-items`. The selector found nothing, so clicking **Our Menu** caused an error and the menu never opened.	Change the selector to `'.menu-items'`.

5. Code Changes

Fix 1: CSS (missing parenthesis)

```
Before:
  background-image: url('https://images.unsplash.com/photo-1514066558159-...%3D%3D';

After:
  background-image: url('https://images.unsplash.com/photo-1514066558159-...%3D%3D');
```

Fix 2: JavaScript (wrong class name)

```
Before:
  const menuItems = document.querySelector('.menu-item');

After:
  const menuItems = document.querySelector('.menu-items');
```

6. Result After the Fixes

After the two corrections, the page displays the wheat-colored welcome box, the tagline, the **Our Menu** button and the café image. Clicking the button shows the four menu items (Breezy Brews, Toasty Delights, Coastal Bites and Sunset Sips), and clicking it again hides them.

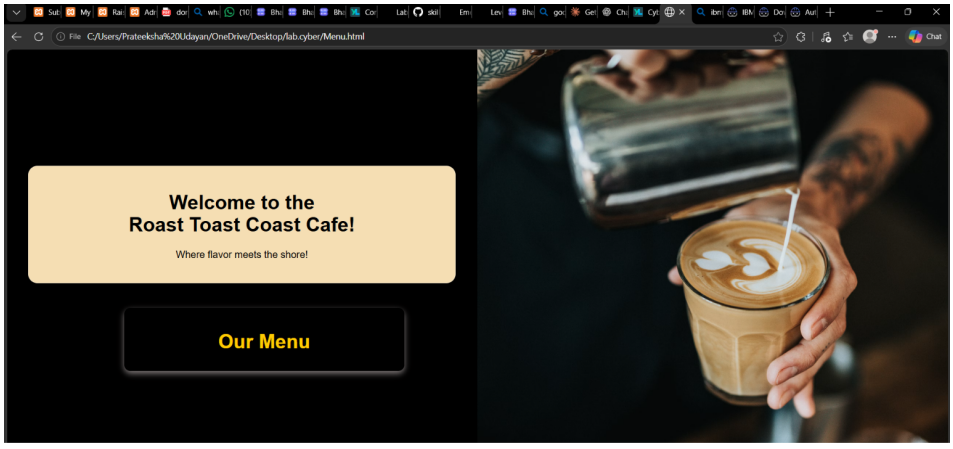


Figure 2: Corrected page, showing the welcome box, the Our Menu button and the image.

7. Additional Observation

The CSS contains rules for `header h1` and `header p`, but the page has no `<header>` element. The heading and tagline are inside `<div class="header-container">`, so those rules never apply and the text shows in the default dark color. This was left unchanged because the dark text is readable on the wheat background. The mismatch could be fixed by changing the selectors to `.header-container h1` and `.header-container p`, or by using a `<header>` element. The gold and white text colors would then need to be changed to stay readable on wheat.

8. Reflection

Selector alignment. The JavaScript selector `.menu-item` did not match the HTML class `menu-items`. Because of the one-letter difference, the script could not find the list, and the button did nothing. Aligning the two names restored the interactive behavior.

Structure and semantics. The unclosed `url(` in the CSS caused the browser to drop the styles that followed it, so styling and element targeting broke across the page. The unused `header` selectors show that CSS rules must match the actual HTML structure for the styling to apply consistently.

Error visibility. Both problems were hard to spot by reading the code. The page showed no error message, only a blank screen, and the wrong class name looked correct until the button was clicked. A categorized review that separates syntax errors from functional errors, with line numbers, makes this kind of problem much easier to find.

Prompt effectiveness. A specific prompt gives more useful results than a general request to fix the code. Describing the symptom (a blank black page and a button that does not work), asking for line numbers and error types, and asking for explanations before any changes makes the output easy to check against what the page actually does.

9. Conclusion

Two small mistakes, a missing closing parenthesis and a mismatched class name, were enough to leave the Roast Toast Coast Café page blank and its menu button unusable. Correcting both restored the layout, the image and the menu toggle. The lab showed that careful comparison between the code and the observed behavior, together with clear, structured prompts, is an effective way to troubleshoot code.